

Reviewer Pack — Minimal Hodge Index and Average-Case Circuit Complexity

Entry 0f549d9f99de · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 20:21:40 UTC

Minimal Hodge Index and Average-Case Circuit Complexity

Entry ID: 0f549d9f99de **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 20:21:40 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Hodge Theory **Field B** (complexity object): Complexity Theory: Average-Case Circuit Complexity

Statement:

[‘For every CNF formula F , the minimal Hodge index $h(F)$ of its associated projective variety is upper-bounded by a polynomial function of its size n , i.e., $h(F) \in O(n^k)$, where k is an absolute constant. Furthermore, for all instances with average-case circuit complexity $C_{\text{avg}}(F) \leq n^l$, the Hodge index of F satisfies $h(F) \geq 2^{1/3} * n^{1/3} + \varepsilon$ for some positive ε .’]

Rationale (proposer’s reasoning):

[‘Hodge theory provides a rich algebraic structure for studying varieties in algebraic geometry. If there exists a correlation between the Hodge index of these varieties and the complexity of their associated CNF formulas, it could reveal new insights into the nature of computational complexity. The proposed conjecture aims to establish such a connection, potentially leading to new algorithms or hardness results.’]

Taxonomy category: AVG_TO_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3df0cceb86086cf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every CNF formula F with $n \leq 40$ variables, if $h(F) \in O(n^k)$, where k is an absolute constant, and for all instances with $C_{\text{avg}}(F) \leq n^l$, $h(F) \geq 2^{1/3} * n^{1/3} + \varepsilon$, then the conjecture is supported. If any seed produces a counterexample with $h(F) > O(n^k)$ or $C_{\text{avg}}(F) > n^l$ but $h(F) < 2^{1/3} * n^{1/3} + \varepsilon$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge index" AND "average-case circuit complexity" - "algebraic geometry" AND "circuit complexity" AND "CNF formula" - "projective variety" AND "polynomial bound" AND "average-case complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below
        factor = Fraction(1, matrix[i][i])
        for j in range(i+1, n):
            matrix[j][i] *= factor

        # Eliminate above
        for j in range(i):
            factor = matrix[j][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    return matrix

def hodge_index(kernel_matrix):
    n = len(kernel_matrix)
    if n == 0:
        return 0
    kernel_matrix = gaussian_elimination(kernel_matrix)
    rank = sum(1 for row in kernel_matrix if any(x != 0 for x in row))
    return rank
```

```

def circuit_complexity(formula, seed):
    random.seed(seed)
    complexity = 0
    for _ in range(100): # Sample 100 instances
        assignment = [random.choice([0, 1]) for _ in range(len(formula))]
        if all(eval(clause, {}, dict(zip('x' + ''.join(str(i) for i in range(len(formula))), assignment)))
            complexity += 1
    return Fraction(complexity, 100)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    formula = [random.choice(['x' + str(i), 'not x' + str(i)]) for i in range(n)]
    kernel_matrix = [[int(eval(clause1, {}, {'x' + str(i): assignment[i] for i in range(n)})) * int(eval(clause2, {}, {'x' + str(i): assignment[i] for i in range(n)})) for i, clause1 in enumerate(formula) for j, clause2 in enumerate(formula))]
    h_index = hodge_index(kernel_matrix)
    avg_complexity = circuit_complexity(formula, seed)

    return {
        "metric_name": "Hodge Index and Circuit Complexity",
        "metric_value": h_index,
        "instances_tested": 100,
        "conjecture_holds": h_index <= n**3 and avg_complexity >= 2**(1/3) * n**(3/3),
        "counterexample": "" if h_index <= n**3 and avg_complexity >= 2**(1/3) * n**(3/3) else f"n={n} h={h_index} avg_complexity={avg_complexity}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5372c865.py", line 79, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5372c865.py", line 61, in run_trial
    kernel_matrix = [[int(eval(clause1, {}, {'x' + str(i): assignment[i] for i in range(n)})) * int(eval(
                                                                ^^^^^^^^^^^
```

NameError: name 'assignment' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14598
2	preregistration	ollama_remote	glm4:latest	0	0	12876
3	novelty	ollama_remote	glm4:latest	0	0	13114
4	novelty	ollama_remote	glm4:latest	0	0	8947
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23750
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9610
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9015
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11213
9	judge	ollama_remote	glm4:latest	0	0	11836

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 114958 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0f549d9f99de.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f549d9f99de.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0f549d9f99de.tar.gz (if generated)